数学与 CS 问答笔记

answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

November 10, 2026

Contents

1. Skill 仓库里有没有专门做 Delve 调试的技能 4
   1.1. 费曼直觉 4
   1.2. 硬核机制 4
      1.2.1. 本次是怎么盘点的 4
      1.2.2. Delve 小节到底覆盖了什么 4
      1.2.3. 磁盘上有，不等于本会话能调用 5
      1.2.4. 依赖其实已经满足 5
   1.3. 陷阱与易错 6
   1.4. 关键点 6
   1.5. 变式检验 6
   1.6. 同类归类 7
   1.7. 本次操作与当前状态 7
2. Delve 调试技巧：golang-troubleshooting 覆盖到哪一层 9
   2.1. 费曼直觉 9
   2.2. 硬核机制 9
      2.2.1. skill 里实际有的四块 9
      2.2.2. 覆盖边界：哪些技巧没有 9
      2.2.3. 技巧一：让断点只在「值得停」的时候停 10
      2.2.4. 技巧二：让调试器自己报告，而不是一直 `next` 11
      2.2.5. 技巧三：watchpoint 抓「谁改了我的变量」 12
      2.2.6. 技巧四：goroutine 太多时先过滤 12
      2.2.7. 技巧五：打印被截断时改配置 12
      2.2.8. 技巧六：脚本化、远程与事后分析 13
      2.2.9. 技巧七：断点设不上，先怀疑优化和内联 13
   2.3. 陷阱与易错 14
   2.4. 关键点 14
   2.5. 变式检验 14
   2.6. 同类归类 15
   2.7. Go 语言机制补充 15
   2.8. 本次操作与当前状态 16
3. 0101 spec 与 strategy 复检：测试绿之后的合同漏洞 17
   3.1. 费曼直觉 17
   3.2. 硬核机制 17
      3.2.1. 判定总览 17
      3.2.2. spec 逐项检查 17
      3.2.3. strategy 逐项检查 20
      3.2.4. 当前测试强度下的 P0 / P1 / P2 21
      3.2.5. 建议的规格骨架（只给问题，不填答案） 21
   3.3. 陷阱与易错 22
   3.4. 关键点 22
   3.5. 变式检验 23

3.6. 同类归类 23
3.7. Go 语言机制补充 24
3.8. 本次操作与当前状态 25
4. 0101 规格框架落地：Go doc comment 骨架 26
4.1. 费曼直觉 26
4.2. 硬核机制 26
4.2.1. 这次改了哪些位置 26
4.2.2. 一个必须修的结构问题 26
4.2.3. 为什么第一行不直接写 TODO 27
4.2.4. 框架里没有写什么 27
4.3. 陷阱与易错 27
4.4. 关键点 28
4.5. 变式检验 28
4.6. 同类归类 28
4.7. Go 语言机制补充 29
4.8. 本次操作与当前状态 29
Index of Listings 31

1. Skill 仓库里有没有专门做 Delve 调试的技能

1.1. 费曼直觉

先给结论：**没有**「只做 Delve」的专用 skill。最接近的是一个综合调试手册 `golang-troubleshooting`，Delve 在里面只占一个小节（约 60 行）。

可以这样类比：你的技能目录像一只家庭工具箱，里面没有「螺丝刀专用箱」，只有「家庭维修箱」；螺丝刀确实在箱子里，但你要先打开维修箱、翻到对应格子。 这和组织方式有关：skill 是**按任务场景**切的（「程序出问题了」），不是**按工具**切的（「我要用 dlv」）。所以问题不是「缺少 Delve 知识」，而是「Delve 知识被收在调试场景下面」。

1.2. 硬核机制

1.2.1. 本次是怎么盘点的

三条命令就能得到全部证据，输出见 Listing 1：

```
    find ~/.reasonix/skills -name SKILL.md | wc -l
    grep -ril delve ~/.reasonix/skills --include='*.md'
    grep -rn -i delve ~/.reasonix/skills --include='*.md' | grep -v golang-
troubleshooting
```

Listing 1: 盘点技能目录与 Delve 关键词分布

结果分三层：

1. **完整教程层**：`~/.reasonix/skills/golang-troubleshooting/references/diagnostic-tools.md` 第 71 行起有 `## Delve Debugger`，包含安装、四种启动方式、常用命令、IDE 配置；
2. **路由/索引层**：`golang-how-to/references/disambiguation.md` 与 `by-category.md` 只写了「调试任务找 `golang-troubleshooting`」，不教 dlv 命令；
3. **一句话提及层**：`golang-popular-libraries/references/tools.md` 只有一行介绍，`software-construction/references/lang-mapping.md` 只有一行「Go 用 dlv」。

1.2.2. Delve 小节到底覆盖了什么

`diagnostic-tools.md` 的 Delve 部分是「速查卡」而非「教材」，骨架如下：

```
go install github.com/go-delve/delve/cmd/dlv@latest

dlv debug ./cmd/myapp
dlv test ./mypackage
dlv attach 12345
dlv exec ./myapp -- --flag=v

break main.main
break file.go:42
continue
next
step
stepout
print variable
locals
args
goroutines
goroutine 5
stack
```

Listing 2: golang-troubleshooting 中的 Delve 命令清单（节选）

它没有讲断点命中后的推理方法、没有讲 `dlv test` 与 `go test` 的编译差异、 也没有讲 DAP/远程调试的坑。这些内容在你的 `note/2026-11-05.typ` 里反而更细（ `dlv test` 的 `--init` 脚本化、 `dlv dap` 的实测输出）。

1.2.3. 磁盘上有，不等于本会话能调用

这次盘点还暴露了一个独立事实：技能目录里有 **61** 个 `SKILL.md`， 而本次会话的可用技能索引只列出 **31** 个； `golang-troubleshooting` 恰好不在那 31 个名字里。因此它现在只能「直接读文件」使用， 不能通过 skill 的自动/手动加载机制触发。

为什么少了 30 个，本次没有定位到原因，只确认了这些事实： `~/.reasonix/config.toml` 的 `[skills]` 段里 `disabled_skills` 是注释状态， 项目目录 `/home/zz/project/db-go/.reasonix` 下只有 `attachments`、没有技能覆盖。要查原因，先看 `reasonix doctor` 的技能加载警告，或临时把 `suppress_warnings` 打开后重开会话。

1.2.4. 依赖其实已经满足

`golang-troubleshooting` 的 frontmatter 声明了运行依赖：

```
requires:
  bins:
    - go
    - dlv
install:
  - kind: go
    package: github.com/go-delve/delve/cmd/dlv@latest
    bins: [dlv]
paths:
  - **/*.go
```

Listing 3: golang-troubleshooting 的依赖与作用路径声明

本机两项都满足：`go` 可用，`dlv` 在 `/home/zz/go/bin/dlv`，版本 `1.27.2`。唯一需要注意的是 `paths: **/*.go`：它意味着这个技能主要面向 Go 源码路径，在 `note/` 里写 Typst 时不会自动命中。

1.3. 陷阱与易错

⚠️ **陷阱预警 三个不能划等号的关系**：① description 里写了 Delve，不等于「专门做 Delve」；② 磁盘上有 `SKILL.md`，不等于当前会话能调用；③ 一行「Go 用 dlv」的提及，不等于调试指南。判断一个 skill 是否真的覆盖某能力，要打开正文（或 `references/`）看有没有 命令、参数、失败模式，而不是只 grep 关键词。

另一个易错点是路径依赖：`golang-troubleshooting` 声明了 `paths: **/*.go`。如果你在讨论 `note/*.typ` 或纯 shell 问题时期待它自动出现，很可能不会触发；这时应该显式读它的文件，而不是等它自己加载。

1.4. 关键点

📌 **核心要点** 当前最接近「Delve 调试技能」的是 `golang-troubleshooting`：它是**综合调试**技能，Delve 是其中一个速查小节。若你只想查 dlv 命令，直接读 `~/.reasonix/skills/golang-troubleshooting/references/diagnostic-tools.md` 最快；若你要的是「调试方法论 + dlv + race + pprof」的组合，它才是正确入口。

技能	Delve 相关内容	深度	本会话索引
`golang-troubleshooting`	专节：安装、4 种启动方式、命令表、IDE 配置	约 60 行	未出现
`golang-popular-libraries`	`tools.md` 一行介绍	一句话	有
`software-construction`	`lang-mapping.md` 一行「Go 用 dlv」	一句话	有
`golang-how-to`	只做路由，指向 `golang-troubleshooting`	索引	有

1.5. 变式检验

变式 1：我想按「dlv 子命令」查，而不是按「调试场景」查。不要走 skill 加载，直接读 `~/.reasonix/skills/golang-troubleshooting/references/diagnostic-tools.md`；或者用 `dlv help` / `dlv help test` 这类离线权威输出。后者永远和本机版本一致，skill 里的命令清单则可能落后于版本。

变式 2：我想补一个真正专用的 Delve skill。可以先用 `find-skills` 或 `install-capability` 搜索技能市场；若市场没有，就自建一个最小技能，只写三样东西：触发条件、「先 `dlv test` 再下断点」的工作流、以及本项目 `internal/db` 的固定命令模板。注意不要把 `SKILL.md` 写成 `dlv help` 的复读机——那部分官方帮助已经足够好，专用技能的价值在于**项目内的工作流与坑**。

变式 3：我现在就要调试 `kv_test.go`。 不需要等技能加载，直接用已有结论：`dlv test ./internal/db -- -test.run TestKVBasic`。更完整的脚本化复现见 `note/2026-11-05.typ` 的「用 Delve 调试 kv_test.go」一节。

1.6. 同类归类

Delve 属于「Go 运行时观测工具链」，和它同类的技能/工具可以这样分：

工具	回答的问题	在本仓库中的载体
`dlv`	程序为什么走到这条分支？变量此刻是什么？	`golang-troubleshooting` 的 Delve 小节
`go test -race`	两个 goroutine 是否在无同步地读写同一块内存？	同上，`concurrency-debug.md`
`pprof`	时间花在哪个函数？谁在分配内存？	`golang-benchmark` / `golang-troubleshooting`
`gopls`	这个符号定义在哪？谁调用了它？	`golang-gopls`

真实项目的做法也印证「工具不是技能」：Delve 官方仓库（`https://github.com/go-delve/delve`）给的是 CLI 与 DAP 协议文档，VS Code Go 扩展给的是 `launch.json` 配置，二者都不以「skill」形式存在。本机安装的 `samber/cc-skills-golang@golang-troubleshooting` 是社区把「调试工作流 + 工具速查」打包成的一份操作手册，差异在于它强调 **先复现、再定位、最后修**，而不是罗列全部 dlv 参数。

1.7. 本次操作与当前状态

已做的文件操作：

1. 搜索 `~/.reasonix/skills`，grep `delve` / `dlv` / `debugger`；
2. 读取 `golang-troubleshooting/SKILL.md` 与 `references/diagnostic-tools.md`；
3. 读取 `golang-how-to/references/disambiguation.md`、`by-category.md`；
4. 检查 `dlv` 是否安装（`/home/zz/go/bin/dlv`，版本 `1.27.2`）；
5. 新建并写入本文件 `note/2026-11-10.typ`（当日首个问题）。

验证结果：

1. `find ~/.reasonix/skills -name SKILL.md | wc -l` → `61`；
2. 本次会话可用技能索引 → `31` 个，`golang-troubleshooting` 不在其中；
3. `dlv version` → `1.27.2`，依赖已满足；
4. `XDG_CACHE_HOME=/home/zz/.cache typst compile 2026-11-10.typ` → 退出码 `0`，无错误；仅有 `ilm` 默认字体 `iosevka` 缺失的环境警告，与历史笔记一致。

需要你自己动手的：

1. 若希望在会话里直接调用 `golang-troubleshooting`，先查清它为什么没进索引（`reasonix doctor` 的技能加载警告最直接）；
2. 若只是要用 Delve，不必等：让代理直接读上面那个 `diagnostic-tools.md` 即可；

3. 若想要真正的专用技能，告诉代理用 `find-skills` 搜索或按上文变式 2 自建。

2. Delve 调试技巧：golang-troubleshooting 覆盖到哪一层

2.1. 费曼直觉

上一问只回答了「有没有专用技能」，这一问要回答「里面的 Delve 内容够不够用」。

先把 Delve 的调试技巧拆成三层，像学开车一样：

1. **知道有这班车**：怎么启动 `dlv debug` / `dlv test`，怎么下断点、怎么单步；
2. **知道在哪停车**：条件断点、命中次数、按 goroutine 过滤，避免在循环里被停几百次；
3. **知道怎么自动记录**：tracepoint、watchpoint、`on`、`display`，让调试器自己报告，而不是人一直盯着。

`golang-troubleshooting` 的第一层写得不错，第二、三层基本没有。所以答案是：**有内容，但只是入门速查，不是技巧手册**。

2.2. 硬核机制

2.2.1. skill 里实际有的四块

位置	内容	层级
`SKILL.md` 第 51 行	升级策略：先用 `fmt.Println` / 测试隔离，再上 pprof、Delve、GODEBUG	方法论
`references/diagnostic-tools.md` 第 71–126 行	安装、四种启动方式、14 个基础命令、VS Code / GoLand 配置	命令速查
`references/methodology.md`	10 步流程、一次一个假设、`git bisect`、卡住后的升级协议	方法论
`references/testing-debug.md`	失败测试、表驱动、`-count` / `-shuffle`、flaky 测试	测试调试，不含 dlv

值得注意的是：`testing-debug.md` 讲了很多「怎么把 bug 缩成一个测试」，却一次也没提 `dlv test`。也就是说，**它教你造现场，但不教你怎么进现场看**。

2.2.2. 覆盖边界：哪些技巧没有

技巧	SKILL 是否覆盖	权威出处
条件断点 `break ... if ...`	否	本机 `help break`；`note/2026-11-05.typ` 已实测
命中次数 / 每 goroutine 计数	否	本机 `help cond` 的 `-hitcount` / `-per-g-hitcount`
命名断点 + `on` 自动执行命令	否	本机 `help break`、`help on`

`display` 每次停下自动打印	否	本机 `help display`
tracepoint（不停下，只报告）	否	本机 `help trace`、`dlv help trace`
watchpoint（监视内存读写）	否	本机 `help watch`
goroutine 过滤 / 分组 / 按 channel 查	否（只列了 `goroutines`、`goroutine` 切换）	本机 `help goroutines`
`config` 调打印上限	否	`pkg/config/config.go` 示例值 64 / 64 / 1；`config -list` 显示未显式定义
`--init` 脚本化复现	否	本机 `dlv help test`；`note/2026-11-05.typ` 已实测
headless / `dlv connect` / DAP	部分（只有 VS Code `launch.json`）	本机 `dlv help dap`、`help connect`
断点保存与加载 `breakpoints -save` + `source`	否	Delve CLI 文档 `Documentation/cli/README.md`
core dump 事后分析	否	本机 `help dump`、`dlv help core`
优化 / 内联导致断点失效	否	编译器 `-N -l`；见下文

结论很清楚：skill 的 Delve 部分是一张「命令菜单」，不是「烹饪技巧」。菜单告诉你 `break` 存在，但没告诉你循环里该用条件断点、该用 hitcount、 还是该把断点变成 tracepoint。

2.2.3. 技巧一：让断点只在「值得停」的时候停

最常用的是条件断点。Delve 的语法是「位置 + `if` + Go 布尔表达式」，不是把表达式塞进位置参数：

```
break kv_test.go:39 if len(val) == 2
cond -hitcount 1 % 100
cond -per-g-hitcount 1 > 3
```

Listing 4: 条件断点、每 100 次命中停一次、每个 goroutine 命中超过 3 次才停

三个工具的适用场景不同：

写法	什么时候停	典型场景
`break ... if expr`	每次到达断点都求值 `expr`，为真才停	只在 `key` 等于某个值时停
`cond -hitcount 1 % 100`	第 100、200、300... 次命中	循环里每 100 次抽样看一次
`cond -per-g-hitcount 1 > 3`	每个 goroutine 各自计数，超过 3 次	并发测试里抓某个 worker

`cond -hitcount 1` 里的 `1` 是断点编号；给断点起名字更好记：

```
break bpGet db.(*KV).Get
cond -hitcount bpGet % 50
```

Listing 5: 命名断点后，条件和自动动作都用名字引用

条件断点不是免费的：每次命中都要在目标进程里求值一次表达式。放在百万次循环里，程序会明显变慢；这时先用 `-hitcount` 过滤，或者把条件写得更便宜（比较长度而不是遍历切片）。

2.2.4. 技巧二：让调试器自己报告，而不是一直 `next`

`on` 把一个断点变成「命中时执行一条命令」的钩子；`display` 则是每次停下都打印。两者组合，可以把「停 100 次、手动 `print` 100 次」变成一次会话：

```
break bpGet db.(*KV).Get
on bpGet print string(key)
on bpGet trace
display -a len(val)
```

Listing 6: 断点命中时自动打印 key；再把它变成不停止的 tracepoint；每次停下都显示 len(val)

`on bpGet trace` 是关键一步：它保留断点位置，但命中时**不暂停**，只打印通知。这等价于一个 tracepoint，但你可以先按普通断点调试，确认位置正确后再「升级」成 tracepoint。

如果一开始就不想停，直接下 tracepoint：

```
trace db.(*KV).Get
trace /Get$/
```

Listing 7: 按函数名或正则批量下 tracepoint；命中只报告，不暂停

`trace` 的参数是 locspec，支持 `/正则/`，所以可以一次追踪一组函数。 命令行还有独立的批量追踪入口：

```
dlv trace -t ./internal/db 'KV.*Get'
```

Listing 8: 不进入交互会话，直接跑测试并报告匹配函数的调用

`dlv help trace` 还提供 `--follow-calls`（追踪子调用到指定深度）、`--stack`（带栈）、`--timestamp`（带时间戳）。输出走 stderr，所以可以 `2> trace.log` 单独收集。

2.2.5. 技巧三：watchpoint 抓「谁改了我的变量」

条件断点回答「值等于 X 时停」，watchpoint 回答「这个内存地址被写时停」。后者适合「值在某个我没预料到的地方被改了」：

```
watch -w count
watch -rw *(*int)(0x1400007c018)
```

Listing 9: 监视变量 count 的写入；或监视一个具体地址的读写

三个限制必须知道：

1. `watch` 的表达式必须**可寻址**。`kv.mem` 是 map header，监视它只能看到 header 被改写，看不到 map 内部元素的增删——map 元素本身不可寻址；
2. 写入相同的值不一定被报告（帮助文档原话：writes that do not change the value might not be reported）；
3. 硬件 watchpoint 数量有限（x86 通常 4 个），不要一次 watch 十几个变量。

2.2.6. 技巧四：goroutine 太多时先过滤

并发测试里 `goroutines` 可能列出几十个，直接 `goroutine 5` 是碰运气。 先按用户代码、运行状态或 channel 过滤：

```
goroutines -with user
goroutines -t 20
goroutines -chan ch
goroutine 12
stack
```

Listing 10: 只看用户 goroutine、带 20 层栈、按 channel 过滤，再切换过去看栈

`-group userloc` 还能按「停在哪个用户函数」分组，快速发现「20 个 goroutine 都堵在同一个 `chan receive` 」。 这正是死锁和 goroutine 泄漏的典型现场。

2.2.7. 技巧五：打印被截断时改配置

Delve 默认把字符串截到 64 字节、结构体只展开少量字段；数组元素和嵌套深度 分别由 `max-array-values` / `max-variable-recurse` 控制。看到 `...` 不是变量真的短， 而是被截断了。 `pkg/config/config.go` 的示例配置写的是 64 / 64 / 1，但 `config -list` 会显示这些项「未显式定义」，此时走内置默认：

```
config max-string-len 10000
config max-array-values 1000
config max-variable-recurse 3
print %x val
whatis val
locals -v
```

Listing 11: 放大打印上限，按十六进制打印，查看类型和带类型的局部变量

这是调试 `[]byte`、长字符串、嵌套结构时最容易被忽略的一步：你以为是数据错了，其实是显示被截断了。

2.2.8. 技巧六：脚本化、远程与事后分析

场景	命令	要点
固定复现步骤	`dlv test ./internal/db --init cmds --test.run TestKVBasic`	命令文件最后写 `exit`
保存 / 恢复断点	`breakpoints -save bps` 与 `source bps`	会话之间复用
记录完整输出	`transcript $TMPDIR/dlv.log`	排查完再 `transcript -off`
远程 / 多客户端	`dlv test ./internal/db --headless --listen=127.0.0.1:40000`	另一个终端 `dlv connect`
IDE 接入	`dlv dap`	VS Code 用 `launch.json` 的 `mode: test`
core dump	`dump $TMPDIR/core`，之后 `dlv core ./db.test $TMPDIR/core`	进程已死也能看现场

非终端环境（CI、管道、脚本）里，交互式 `dlv` 必须加 `--allow-non-terminal-interactive=true`，否则 Delve 会直接拒绝启动。这一点 `note/2026-11-05.typ` 已经实测过。

2.2.9. 技巧七：断点设不上，先怀疑优化和内联

`dlv test` 和 `dlv debug` 会自动给编译器加 `-gcflags=all=-N -l`：`-N` 关优化，`-l` 关内联。`dlv exec` 调试的是**已经编译好**的二进制，不会替你加这两个标志。所以：

```
go test -c -gcflags='all=-N -l' -o $TMPDIR/db.test ./internal/db
dlv exec $TMPDIR/db.test -- -test.run TestKVBasic
```

Listing 12: 先自己关优化编译测试二进制，再用 dlv exec 调试

如果断点显示 `Breakpoint set` 但永远不会命中，按这个顺序排查：

1. 代码到底有没有被执行（先在函数入口下断点）；
2. 二进制是否带调试信息（`-ldflags='-s -w'` 会剥离 DWARF，Delve 读不到变量）；
3. 是否被内联（`-l` 关掉）；
4. 文件名是否有歧义（用完整路径或包限定名）；
5. `init` 函数是否已经跑完（`main.init` 要在启动阶段下断点）。

2.3. 陷阱与易错

⚠️ **陷阱预警 Delve 的五个常见坑：** ① 条件断点必须用 `if` 引出，写成 `break func expr` 会被当成位置名；② `print` 默认截断 64 个元素 / 64 字节，看到 `...` 先调 `config`；③ `watch` 要求可寻址，map 元素不可寻址，同值写入可能不报告；④ `dlv exec` 不会自动关优化，断点可能漂移或设不上；⑤ `trace` 输出到 stderr，且会一直跑到程序结束，长驻程序要配超时。

还有一个「思维坑」：Delve 能看到现场，不代表应该一开始就用它。`golang-troubleshooting` 的升级策略是对的——先 `fmt.Println`、先写失败测试、先跑 `-race`、先看 pprof；只有在「必须知道某一刻的局部变量和调用栈」时才上 Delve。用 Delve 去查一个用 `-race` 五秒就能定位的数据竞争，是负收益。

2.4. 关键点

📌 **核心要点** `golang-troubleshooting` 的 Delve 内容只覆盖第一层：四种启动方式 + 14 个基础命令 + IDE 配置。真正拉开效率差距的是第二、三层：条件断点 / hitcount、`on` + `display`、tracepoint、watchpoint、goroutine 过滤、`config` 打印上限、`--init` 脚本化。这些内容它没有，但都可以从本机 `help <命令>` 和 `Documentation/cli/README.md` 查到。

层次	代表技巧	SKILL 覆盖	本项目怎么用
第一层：能停	`break` / `next` / `step` / `print`	有	`break kv_test.go:39`
第二层：停得准	条件断点 / hitcount / goroutine 过滤	无	`cond -hitcount bpGet % 50`
第三层：自动记录	`on` / `display` / `trace` / `watch`	无	`on bpGet trace`

2.5. 变式检验

变式 1：条件断点、hitcount、per-goroutine hitcount 怎么选？条件依赖**变量值**时用 `if`；只关心「第 N 次」时用 `-hitcount`；并发下每个 goroutine 各自计数时用 `-per-g-hitcount`。如果条件表达式很贵（比如遍历大切片），先用 hitcount 降频，再在停下来时手动检查。

变式 2：断点、tracepoint、watchpoint 的区别？ 断点**暂停**程序，适合交互式检查；tracepoint **不暂停**，只报告命中，适合观察调用序列；watchpoint **监视内存地址的读写**，适合抓「谁改了这个值」。三者可以组合：先下断点确认位置，再 `on <bp> trace` 升级成 tracepoint。

变式 3：断点设不上怎么办？ 先确认代码执行到（函数入口断点），再确认二进制没被 `-s -w` 剥符号，再确认没被内联（`-l`），最后检查文件名/函数名是否歧义。`dlv exec` 的用户尤其容易漏掉 `-N -l`。

变式 4：什么时候不该用 Delve？ 能用日志/断言复现时先用日志；数据竞争先用 `go test -race`；性能问题先用 pprof；崩溃先用 `GOTRACEBACK=all`。Delve 的不可替代场景是「必须知道某一刻的局部变量、调用栈、goroutine 状态」。

2.6. 同类归类

工具	优势	局限
`dlv`	理解 Go 运行时：goroutine、channel、DWARF、DAP	需要调试信息；优化代码需 `-N -l`
`gdb` / `lldb`	通用、支持多语言、汇编级强	不理解 goroutine 调度和 Go 类型
日志 / `fmt.Println`	零依赖、可进生产	要改代码；无法看历史状态
`pprof` / `-race` / `GODEBUG`	低开销、适合性能与并发问题	不提供单步和局部变量

真实项目的两套用法可以对照：

1. **Delve 官方 CLI 文档**（本机 `~/go/pkg/mod/github.com/go-delve/delve@v1.27.2/Documentation/cli/README.md`）按**命令**组织：`break` 在第 109 行、`condition` 在第 206 行、`on` 在第 563 行、`trace` 在第 741 行、`watch` 在第 788 行。它给的是完整语法和边界条件。
2. **VS Code Go 扩展**走的是另一条路：IDE 不直接拼 CLI 命令，而是启动 `dlv dap`，用 `launch.json` 的 `request` / `mode` / `program` / `args` 描述会话。同一个调试器，一个是「命令式」，一个是「配置式」。

本项目的选择是 CLI + `--init`：因为 `internal/db` 的测试失败需要可复现、可粘贴、可写进笔记的命令，而不是一份只在我机器上点得动的 IDE 配置。`note/2026-11-05.typ` 第 940 行起的那一节，就是这条路线跑通后的记录。

2.7. Go 语言机制补充

`-N` 与 `-l` 到底关了什么。 `-N` 关闭优化，`-l` 关闭内联。内联会把小函数的指令直接嵌进调用者，源码行和机器指令不再一一对应，断点可能落到调用者、被合并，甚至完全设不上。`dlv test` / `dlv debug` 自动加这两个标志；`dlv exec` 不会。标志定义在 `/usr/lib/go/src/cmd/compile/internal/base/flag.go`，`-N` 在第 64 行、`-l` 在第 74 行附近。

DWARF 是 Delve 的「地图」。 Go 编译器默认生成 DWARF 调试信息，记录变量名、类型、源码行与指令地址的映射。`-ldflags='-s -w'` 会剥离符号表和 DWARF，二进制更小，但 Delve 就读不到变量和行号了。生产二进制常这么干，所以「线上 core dump 用 Delve 分析」通常需要一份未剥离的对应二进制。

goroutine 不是 OS 线程。 Delve 通过 ptrace 控制 OS 线程，再借助 Go 运行时的 `allgs` 等符号枚举 goroutine。`goroutines` 列的是**当前暂停时刻**的快照；切换 goroutine 只是切换「当前查看的栈和变量上下文」，不会让那个 goroutine 单独运行。

ptrace 权限。 Linux 上 `dlv attach` 需要 ptrace 权限。容器里常见 `/proc/sys/kernel/yama/ptrace_scope` 限制，表现为 `could not attach to pid`。`dlv test` / `dlv debug` 是自己启动子进程，通常不受此限制。

DAP 是协议，不是工具。 `dlv dap` 启动一个 TCP 服务，VS Code 作为 DAP 客户端 发送「下断点 / 继续 / 看变量」请求。命令行里的 `break` / `continue` / `print` 和 IDE 里的按钮，最终都落到同一套调试会话操作上。

2.8. 本次操作与当前状态

已做的文件操作:

1. 读取 `golang-troubleshooting/SKILL.md` 第 160–190 行与 `references/diagnostic-tools.md` 全文；
2. 读取 `references/methodology.md` 的升级协议、`references/testing-debug.md` 全文；
3. 用 `dlv test ./internal/db --init ...` 查询 `help break` / `help cond` / `help on` / `help watch` / `help trace` / `help display` / `help goroutines` / `help dump` / `help config` 的本机输出；
4. 读取本机 Delve 源码 `pkg/config/config.go` 的打印配置示例值与 `Documentation/cli/README.md` 的命令索引；
5. 追加本一级标题到 `note/2026-11-10.typ`。

验证结果:

1. `dlv version` ⟶ `1.27.2`；
2. 帮助输出确认了 `break [name] [locspec] [if <condition>]`、`cond -hitcount` / `-per-g-hitcount`、`on <bp> trace`、`watch -r|-w|-rw`、`trace [name] [locspec]`、`display -a` 等语法；
3. `pkg/config/config.go` 示例配置写有 `max-array-values: 64`、`max-string-len: 64`、`max-variable-recurse: 1`；`config -list` 显示三项 `<not defined>`，说明运行时走 Delve 内置默认；
4. `XDG_CACHE_HOME=/home/zz/.cache typst compile 2026-11-10.typ` ⟶ 退出码 `0`，无错误；仅有 `ilm` 默认字体 `iosevka` 缺失的环境警告，与历史笔记一致。

需要你自己动手的:

1. 想在项目里练手，从 `dlv test ./internal/db -- -test.run TestKVBasic` 开始，先试 `break kv_test.go:39 if len(val) == 2`，再试 `on bpGet print string(key)`；
2. 如果断点设不上，先改用 `dlv test`（自动 `-N -l`），确认不是优化问题；
3. 长字符串/大切片打印不全时，先 `config max-string-len 10000`，不要急着怀疑数据；
4. 需要 IDE 时用 `dlv dap` + VS Code 的 `launch.json`，命令行和 IDE 可以随时切换。

3. 0101 spec 与 strategy 复检：测试绿之后的合同漏洞

3.1. 费曼直觉

测试已经 PASS，`go vet` 也没有输出。但测试绿只说明**施工通过了当前这张验收单**，不说明**合同写完整了**、也不说明**验收单覆盖了合同**。

把 0101 拆成三份文件看：

1. `kv.go` 顶部注释 = 合同（AF / RI / Safety）；
2. `kv.go` 五个方法的 doc comment = 每个操作的条款；
3. `kv_test.go` 顶部 strategy + 断言 = 验收范围。

现在的状态是：合同缺了「每个方法到底承诺什么」这一章；验收范围里还有三个 `TODO`；施工里有一个开关（`Set` 的 `updated`）永远停在 `true`，而验收单只在新键上按过它一次。所以测试是绿的，但合同和验收范围都还没补全。

> 📌 **核心要点** 当前测试强度下，最贵的不是「再加测试」，而是把**返回值语义**写进合同：`Get` 的 `ok`、`Set` 的 `updated`、`Del` 的 `deleted`、`Open` / `Close` 的生命周期。这四件事不定，测试写多少条都只是碰运气。

3.2. 硬核机制

3.2.1. 判定总览

对象	现状	判定	当前强度下的动作
测试	`go test ./internal/db/ -count=1` 通过；`go vet` 无输出	入场券	不改测试迁就实现
AF	有一行集合构造式	基本可用，缺定义域	补一句抽象域与键的对应
RI	两条长度约束 + 空 `checkRI`	与实现矛盾	改成当前实现真正维护的约束
Safety	一句话说 Get / Set 拷贝	方向对，缺分向	补入向 / 出向 / 未命中
方法规格	五个方法都没有 doc comment	缺	P0：每条写 3–5 行
strategy	三个 `TODO`	未补全	P0：改成覆盖清单
`Set` 的 `updated`	恒为 `true`	契约未定	P0：定语义并对齐实现
删除后的断言	第 50 行查的是 `xxx`	没验证删除	P0：改成查 `k1`

3.2.2. spec 逐项检查

AF 现在写的是：

```
AF(mem) = { k -> mem[k] | all k in mem }
```

Listing 13: kv.go 第 10 行的 AF

它能表达「mem 里每个键映射到它的值」，但还缺三句话：

1. 抽象域是什么：从字节串到字节串的**有限映射**；
2. 内部键是 `string`，抽象键是 `[]byte`；`string(key)` 之后，**字节内容相同即同一个键**；
3. `mem` 为空时，抽象值是空映射，不是「未定义」。

这三句不是学术装饰。没有第二句，客户不知道 `[]byte{0x6b, 0x31}` 和另一个内容相同的切片是不是同一个键；没有第三句，`Open` 之后立刻 `Get` 的行为就没有规格依据。

RI 现在写的是：

```
len( each of mem.keys) >= 1
len( each of mem.values) >= 1
```

Listing 14: kv.go 第 13–14 行的 RI

这里有三个独立问题：

问题	为什么	怎么补
语法	`mem.keys` 在 Go 里不存在；`mem` 只有键类型 `string` 和值类型 `[]byte`	写成 `for all k in dom(mem): len(k) >= 1` 这类可判真假的谓词
与实现矛盾	`Set` 没有任何拒绝空键 / 空值的分支，空键和空值都能写进去	要么让 `Set` 拒绝并定义错误，要么删掉这两条
`mem` 非 nil	零值 `KV{}` 的 `mem` 是 nil；写 nil map 会 panic	补 `mem != nil`，或明确「必须先 Open」

第三条上次评审已经提过，现在仍未补。`checkRI` 也还是空壳：

```
// func (kv *KV) checkRI() {
//
//
// }
```

Listing 15: kv.go 第 29–32 行：空的 checkRI

空壳比没有更糟：读者会以为「RI 有代码在检查」，实际上没有任何调用点。 当前测试强度下，要么实现一个最小的 `checkRI`（检查 `mem != nil` 与空键 / 空值）， 要么删掉空壳，只在注释里写「RI 由构造与写入路径维护」。

Safety 的方向是对的，但只写了「Get 和 Set 做防御性拷贝」。需要拆成三句：

1. **入向**：`Set` 存的是 `bytes.Clone(val)`，调用方之后修改 `val` 不影响 rep； `key` 通过 `string(key)` 转换，字节被复制，调用方之后修改 `key` 也不影响 rep；

2. **出向**：`Get` 返回 `bytes.Clone(val)`，客户修改返回值不影响 rep；
3. **未命中**：`Get` 返回 nil 切片（`bytes.Clone(nil)` 就是 nil），不是 rep 内部切片；`Del` 和 `Close` 不返回内部引用。

这三句里，第三句直接影响客户怎么写代码：`val == nil` 和 `len(val) == 0` 在未命中时是同一件事，但客户如果拿 `val` 去 `append`，nil 和空切片的行为不同。

方法规格是最大的空白。现在五个方法都没有 doc comment，合同只活在测试断言里：

方法	必须回答的问题	当前测试是否要求
`Get`	未命中时 `val` 是 nil 还是空切片？`ok` 与 `err` 各表达什么？	只要求 `!ok && err == nil`
`Set`	`updated` 何时 true / false？同键同值算不算更新？空值呢？	只在新键上要求 true
`Del`	`deleted` 表示「键存在」还是「删除成功」？重复删除呢？	未命中 / 命中各一条
`Open`	调用前对象可用吗？重复调用会清空吗？	只调用一次
`Close`	关闭后还能用吗？会失败吗？	defer 调用，未检查返回值

`Set` 的 `updated` 现在有一个具体漏洞。看实现：

```
1    kv.mem[string(key)] = bytes.Clone(val)
2    newVal, _ := kv.mem[string(key)]
3    updated = bytes.Equal(val, newVal)
```

Listing 16: kv.go 第 49–51 行：先写，再拿刚写进去的值和入参比较

`newVal` 是 `val` 的克隆，内容必然相同，所以：

```
1    bytes.Equal(val, newVal) == true   // 恒真，包括 val 为 nil 的情况
```

Listing 17: 写入后比较恒真；updated 永远为 true

当前测试只在新键上断言 `updated` 为 true，所以它能过。若合同按讲义写「同键同值返回 false」，这个实现就是错的；若合同写「只表示写入动作发生」，它又和字段名 `updated` 的字面意思冲突。当前测试强度下不必立刻加同键同值用例，但**合同必须选一条**，否则实现可以永远返回 true。

生命周期也是空白。`Open` 把 `mem` 初始化为空 map，`Close` 直接返回 nil；但注释没有说：

1. `KV{}` 的零值能不能直接 `Set`？实测不能，会 `panic: assignment to entry in nil map`；
2. `Open` 重复调用会不会清空已有数据？实现会；
3. `Close` 之后还能不能 `Get` / `Set`？实现可以，因为没有状态；
4. `Open` / `Close` 返回的 `error` 什么时候非 nil？实现永远 nil。

这些不是「未来功能」，而是当前五个方法已经存在的行为。合同不写，客户只能读实现。

3.2.3. strategy 逐项检查

`kv_test.go` 顶部现在有三个 `TODO`（第 9、17、20 行），它们本身就是「未补全」的证据：

```
partition on operation:
  Get: use key state
  Set: TODO（分叉点：键是否已存在 / 值是否变化）
  Del: use key state
partition on history（调用历史）:
  TODO
当前未覆盖的子域（阶段末再决定是否加测）:
  TODO
```

Listing 18: kv_test.go 第 7–20 行的 strategy 现状

逐项看覆盖：

分区	已有用例	覆盖	缺口
`Get` / key 存在	第 38 行	命中	未断言 `val` 是副本
`Get` / key 不存在	第 41、50 行	未命中	两次都是 `xxx`，重复
`Set` / 新键	第 35 行	`updated == true`	没有同键同值 / 同键不同值
`Del` / 不存在	第 44 行	`deleted == false`	没有重复删除
`Del` / 存在	第 47 行	`deleted == true`	删除后没有查 `k1`
history	`Set` 后 `Get`	一条链	没有 `Del` 后 `Get`、`Open` 后 `Get`、`Close` 后
Safety	无	无	入向 / 出向拷贝都没测
边界	无	无	空键 / 空值 / nil

最具体的一处是第 47–51 行：

```
updated, err = kv.Del(k1Key)
assert.True(t, updated && err == nil)

_, ok, err = kv.Get(xxxKey)
assert.True(t, !ok && err == nil)
```

Listing 19: kv_test.go 第 47–51 行的结构：删除 k1，却回头查 xxx

删除 `k1` 之后，测试查的是 `xxx`。`xxx` 从来就没存在过，所以这条断言即使删除逻辑完全没执行也会通过。它没有验证「`k1` 已经不可见」这个删除的核心效果。当前测试强度下，把 `xxx` 改成 `k1` 就能补上，不需要新增测试函数。

strategy 还缺一个「当前未覆盖子域」的清单。三个 `TODO` 不应继续留着，而应改成两类：

1. **当前已覆盖**：`Get` 命中 / 未命中、`Set` 新键、`Del` 命中 / 未命中、`Set` 后 `Get`；

2. **阶段末再补**：`Set` 同键同值 / 不同值、Safety 入向 / 出向、空键 / 空值、`Open` / `Close` 生命周期、 重复 `Del` 、子测试与断言拆分。

这不是「现在就加测试」，而是把「不知道缺什么」变成「知道缺什么、决定什么时候补」。按你设定的渐进强度，阶段末清单可以保留 `TODO(阶段末)` 标记，但不能是空 `TODO` 。

3.2.4. 当前测试强度下的 P0 / P1 / P2

优先级	动作	判据	是否新增测试
P0	写 `Get` / `Set` / `Del` / `Open` / `Close` 的方法规格	每个返回值都有语义	否
P0	定 `Set` 的 `updated` 语义并对齐实现	同键同值的行为有明确说法	否
P0	修 RI： `mem != nil` ； 空键 / 空值二选一	RI 与 `Set` 行为一致	否
P0	把 strategy 三个 `TODO` 改成覆盖清单	已覆盖 / 阶段末分清	否
P0	第 50 行 `Get` 改成 `k1`	删除后确实查不到	否，只改一行
P1	`Set` 同键同值 / 不同值用例	覆盖 `updated` 的两个分支	是，阶段末
P1	Safety 入向 / 出向拷贝用例	改返回值 / 改入参后 rep 不变	是，阶段末
P1	空键 / 空值边界	与 RI 的最终选择一致	是，阶段末
P1	`Open` / `Close` 幂等与生命周期	零值 / 重复 Open / Close 后有断言	是，阶段末
P2	并发安全、持久化、后续 step 接口	不属于 0101 当前测试	否，不要提前做

P0 里只有最后一件会动测试，而且只是把一个键改对；其余四件都是注释和契约。这正是「当前测试强度下」该做的事：不扩大验收范围，只把合同和验收范围写清楚。

3.2.5. 建议的规格骨架（只给问题，不填答案）

`Get` 的 doc comment 至少要回答：

1. `val` 在命中时是什么？未命中时是 nil 还是空切片？
2. `ok` 与 `err` 的分工是什么？未命中算不算错误？
3. 返回的 `val` 与 rep 是否共享底层数组？

`Set` 的 doc comment 至少要回答：

1. `updated` 为 true 的条件是什么？false 的条件是什么？
2. 同键同值算不算更新？空值算不算合法值？
3. 入参 `key` / `val` 在调用后还能不能被调用方修改？

`Del` 的 doc comment 至少要回答：

1. `deleted` 表示「键存在」还是「删除成功」？
2. 重复删除同一个键返回什么？
3. 删除不存在的键算不算错误？

`Open` / `Close` 的 doc comment 至少要回答：

1. 零值 `KV{}` 能不能直接使用？必须先 `Open` 吗？
2. 重复 `Open` 会不会清空？`Close` 之后还能不能用？
3. 返回的 `error` 什么时候非 nil?

这些问题的答案由你定，但同一份合同里只能有一套答案。测试强度可以渐进，合同不能渐进——今天写「未命中返回 nil」，明天测试就不能要求空切片。

3.3. 陷阱与易错

⚠️ **陷阱预警 测试绿不等于合同完整。** 当前 `Set` 的 `updated` 恒为 true，测试仍然通过， 因为测试只在新键上按了一次这个开关。绿只证明「当前用例没抓到」， 不证明「行为符合讲义」。

⚠️ **陷阱预警 写了不维护的 RI，比不写更危险。** RI 说键和值长度至少 1，但 `Set` 允许空键 / 空值。客户读到 RI 会假设空键会被拒绝；实现却默默接受。要么实现拒绝，要么删掉这两条， 不能让它停留在「注释里是真的、代码里是假的」。

⚠️ **陷阱预警 `TODO` 不是测试策略。** `Set: TODO` 和 `history: TODO` 让读者无法判断「这是已知缺口，还是忘了写」。阶段末清单可以保留 `TODO(阶段末)` ， 但必须写清缺什么、为什么现在不补。

⚠️ **陷阱预警 删除后查错键，是最隐蔽的假绿。** 第 50 行查 `xxx` ，而删除的是 `k1` 。这条断言在删除逻辑完全失效时也会通过；它验证的是「不存在的键仍然不存在」，不是「删除生效」。

3.4. 关键点

📌 **核心要点** 当前测试强度下，0101 的 P0 只有五件：补五个方法规格、定 `updated` 、修 RI、把 strategy 的 `TODO` 变成覆盖清单、把第 50 行的 `xxx` 改成 `k1` 。前四件不改测试，最后一件只改一个键。做完这些，测试仍然是绿的， 但合同、验收范围、实现三者才对得上。

层次	当前状态	完成判据
Apply	测试 PASS，vet 绿	已满足
Explain	方法规格缺失，`updated` 未定义	每个返回值都能说出语义
Predict	未覆盖同键同值、空键、Close 后	能预测这些边界的行为

Discriminate	未命中 vs 错误、nil vs 空切片未写	能区分并写进合同

按 skill 的掌握判据：现在最多是通过（Apply），不能记内化；因为 Explain 和 Predict 还没有证据。

3.5. 变式检验

变式 1：如果 `updated` 的定义是「值是否发生变化」，当前实现哪一行错？ 先写后比，`newVal` 是刚写入的克隆，所以 `bytes.Equal` 恒真。正确顺序是先读旧值、再比较、再写入；同键同值返回 false，同键不同值和新键返回 true。

变式 2：如果 RI 保留「键和值长度至少 1」，`Set` 应该怎么改？ `Set` 必须在写入前检查 `len(key) >= 1` 和 `len(val) >= 1`，不满足时返回错误；同时要定义错误值，并补空键 / 空值的测试。当前测试强度下可以不补测试，但实现和 RI 必须同时改，不能只改注释。

变式 3：如果 Safety 改成「声明别名、不拷贝」，哪些注释和测试要改？ Safety 要写清「返回的切片在下次修改前有效」，`Get` 不再 `Clone`；测试要验证「修改返回值会影响后续读取」——这与当前「拷贝」合同正好相反。两条路线都合法，但不能一份合同写拷贝、实现做别名。

变式 4：如果 `Close` 之后对象不可用，当前实现要怎么改？ `Close` 需要把 `mem` 置为 nil 或增加 `closed` 标志，`Get` / `Set` / `Del` 在关闭后返回错误；测试要覆盖「关闭后调用失败」。当前实现是 no-op，所以合同要么写「Close 后仍可用」，要么改实现。按当前测试强度，写清楚现状即可，不必现在加关闭状态。

3.6. 同类归类

设计点	常见路线	本项目的选择与出处
未命中	`ok bool` / 返回 nil / 返回 error	当前选 `ok bool`；`sync.Map.Load` 同路线
拷贝	入向拷贝 / 出向拷贝 / 声明别名	当前选双向拷贝；`bytes.Clone` 兑现
生命周期	零值可用 / 必须 Open	当前必须 Open；`os.File` 同路线，但合同未写
测试组织	单函数冒烟 / 表驱动子测试 / ADT 序列	当前单函数冒烟；阶段末再升级

真实项目的两个样本：

1. `sync.Map.Load` 的文档把「未命中」写成 `ok`，不产生 error：

```
// Load returns the value stored in the map for a key, or nil if no
// value is present.
// The ok result indicates whether value was found in the map.
func (m *Map) Load(key any) (value any, ok bool)
```

Listing 20: go doc -src sync.Map.Load：未命中不是错误

1. `bytes.Clone` 的文档把「nil 进、nil 出」写死：

```
// Clone returns a copy of b[:len(b)].
// The result may have additional unused capacity.
// Clone(nil) returns nil.
func Clone(b []byte) []byte {
  if b == nil {
    return nil
  }
  return append([]byte{}, b...)
}
```

Listing 21: go doc -src bytes.Clone：未命中的 nil 语义可以直接引用

你的 `Get` 已经在行为上采用了这两条路线，但合同里没有写。样本的价值不是「照抄」，而是让你看到：**行为、文档、测试三处说的是同一件事**。

3.7. Go 语言机制补充

`string([]byte)` 会复制字节。 Go 的字符串不可变，`string(key)` 产生一个新的字符串值，调用方之后修改 `key` 不会影响 rep。所以 Safety 里 `key` 不需要再 `Clone`，但要把这条理由写出来，否则读者会问「为什么 `val` 拷贝了，`key` 没拷贝」。

`bytes.Clone(nil)` 返回 nil。 这是 `Get` 未命中时 `val == nil` 的直接原因。如果合同写「未命中返回空切片」，实现就要改成 `[]byte{}`；如果写 nil，现在的实现已经符合。两种都可以，但必须选一条并让测试断言。

`bytes.Equal(nil, []byte{})` 返回 true。 这意味着即使 `val` 是 nil，`Set` 里「写后比较」也会得到 true。`updated` 恒真的结论对 nil 也成立。

nil map 的三种操作。 读 nil map 合法，返回零值；`delete` nil map 是 no-op；写 nil map 会 `panic: assignment to entry in nil map`。所以 `KV{}` 上 `Get` / `Del` 可能不 panic，`Set` 一定 panic。生命周期合同要覆盖这个差异，而不是笼统写「必须先 Open」。

`assert.True` 的失败信息只有一句。 `assert.True(t, a && b && c)` 失败时只输出 `Should be true`，三个条件哪个错、`val` 是什么都看不到。当前测试强度下可以不改，但阶段末拆成 `require.NoError` + `assert.True` + `assert.Equal`，失败定位会便宜很多。`require` 失败调用 `t.FailNow()`，适合「后续断言依赖这一步成功」的场合（如 `Open`）。

6.031 对照。 0101 对应 R06 规格（前置 / 后置条件）、R11 AF/RI、R08 不可变。R06 的核心正是「未命中是正常返回值还是错误」这类契约选择；R11 要求 AF 与 RI 对每个合法 rep 都能判真假；R08 解释为什么 `[]byte` 要拷贝。三条都能在当前文件里找到落点。

3.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv.go`（61 行）、`internal/db/kv_test.go`（54 行）；
2. 读取 `note/2026-11-07.typ` 的 0101 问题清单与后续建议；
3. 加载 `software-construction` skill，按其「规格 / 测试」动作与质量门槛评审；
4. 追加本一级标题到 `note/2026-11-10.typ`；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go test ./internal/db/ -run TestKVBasic -v -count=1` → `PASS`；
2. `go test ./internal/db/ -count=1` → `ok`（全包通过）；
3. `go vet ./internal/db/` → 无输出；
4. `go doc -src bytes.Clone` → 确认 `Clone(nil) returns nil`；
5. `go doc -src sync.Map.Load` → 确认未命中用 `ok`、不产生 error；
6. 静态推理确认 `Set` 的 `updated` 恒为 true：`newVal` 是 `val` 的克隆，`bytes.Equal(val, newVal)` 必然为 true，包括 `val` 为 nil 的情况；
7. `XDG_CACHE_HOME=/home/zz/.cache typst compile 2026-11-10.typ` → 退出码 `0`，无错误；仅有 `ilm` 默认字体 `iosevka` 缺失的环境警告。

需要你自己动手的：

1. P0：给 `Get` / `Set` / `Del` / `Open` / `Close` 写 doc comment，每条回答「建议的规格骨架」里的问题；
2. P0：定 `Set` 的 `updated` 语义；若按讲义「同键同值 false」，改实现为先读旧值再比较；
3. P0：修 RI：补 `mem != nil`；空键 / 空值要么让 `Set` 拒绝，要么删掉这两条；
4. P0：把 `kv_test.go` 的三个 `TODO` 改成「已覆盖 / 阶段末」清单；
5. P0：把第 50 行的 `Get([]byte("xxx"))` 改成 `Get([]byte("k1"))`，验证删除效果；
6. P1（阶段末）：补 `Set` 三分区、Safety 两例、空键 / 空值、`Open` / `Close` 生命周期；
7. 不要现在做：并发安全、持久化、后续 step 的接口扩展。

4. 0101 规格框架落地：Go doc comment 骨架

4.1. 费曼直觉

Go 里的「规格」不是另写一份文档，而是 `go doc` 能读到的注释。所以「创建框架」不是往文件里塞一段说明，而是把**注释放在正确的位置、用正确的形状**，让 `go doc` 能把摘要和条款显示出来；具体条款留成 `TODO(user)`，由你填。

可以把它想成填表：表格的栏目已经画好，第一栏必须写方法名，后面几栏是 前置条件、返回值、错误、别名；现在每栏里放的是问题，不是答案。

4.2. 硬核机制

4.2.1. 这次改了哪些位置

位置	加了的框架	留给你的内容
`KV` 类型注释	AF / RI / Safety 三段各加一个 `TODO(user)`	抽象域、键对应、空 map、RI 谓词、`mem != nil`、入向 / 出向拷贝
`Open` / `Close`	生命周期 doc comment 骨架	零值可用性、重复调用、关闭后行为、error 条件
`Get` / `Set` / `Del`	返回值语义 doc comment 骨架	未命中、`updated`、`deleted`、空键 / 空值、别名
`checkRI`	注释块里加 `TODO(user)`	实现检查，或删掉这个钩子

4.2.2. 一个必须修的结构问题

原来的 `// var (...)` 注释块夹在 `KV` 的类型注释和 `type KV struct` 之间。Go 的 doc comment 必须**紧邻**它描述的声明；中间夹了别的注释或空行，`go doc` 就不会把这段注释关联到 `KV`。本次把 `// var (...)` 移到 `type KV` 之后，类型注释才出现在 `go doc` 里：

```
1    type KV struct {
2      // Has unexported fields.
3    }
4        KV is a mutable in-memory key-value store.
5
6            Abstraction function:
7              AF(mem) = { k -> mem[k] | all k in mem }
8
9        TODO(user): state the abstract domain, how string keys map to byte
keys,
10       and what the empty map denotes.
```

Listing 22: go doc ./internal/db KV 的输出：类型规格已经可见

方法注释同理。每个方法的第一行都以方法名开头，`go doc` 才会把它当作摘要：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error)
    Get returns the value associated with key.

    TODO(user): describe the precondition on key.
    TODO(user): describe val and ok when key is present.
    TODO(user): describe val and ok when key is absent.
    TODO(user): state when err is non-nil.
    TODO(user): state whether val aliases the representation.
```

Listing 23: go doc ./internal/db KV.Get 的输出：方法规格已经可见

4.2.3. 为什么第一行不直接写 TODO

Go doc 的第一行是摘要。如果第一行写 `TODO(user): ...`，`go doc` 的摘要就变成一句 没有信息量的待办。所以框架里保留了一句**中性摘要**（`Get returns the value associated with key.`），它只描述方法的基本动作，不替你决定未命中、`updated`、`deleted` 这些契约。你可以改这句摘要，但改完仍要让第一行以方法名开头。

4.2.4. 框架里没有写什么

1. 没有写 `Get` 未命中时返回 nil 还是空切片；
2. 没有写 `Set` 的 `updated` 在同键同值时是 true 还是 false；
3. 没有写 `Del` 的 `deleted` 是「键存在」还是「删除成功」；
4. 没有写 `Open` 前能不能直接用、`Close` 后还能不能用；
5. 没有写 RI 的空键 / 空值到底合不合法。

这些正是 `TODO(user)` 要你回答的问题。规格的价值不在「写满」，而在「选一条并保持一致」。

4.3. 陷阱与易错

⚠ **陷阱预警 doc comment 与声明之间不能夹东西。** 原来的 `// var (...)` 块夹在 `KV` 注释和 `type KV` 之间，导致 `go doc ./internal/db KV` 完全不显示 AF / RI / Safety。只加注释、不检查 `go doc`，很容易以为框架已经生效。

⚠ **陷阱预警 第一行不以符号名开头，摘要就不对。** `// This method gets ...` 不会让 `go doc` 把 `Get` 的摘要排好；写成 `// Get returns ...` 才是 Go 的惯例。也不要用 `@param` / `@return`，Go 的文档注释没有这些标签。

⚠ **陷阱预警 `TODO(user)` 不是答案。** 它只是占位。填的时候必须选一条契约，并同步检查测试；如果测试与契约冲突，先改契约和测试，不要让实现去迁就一个含混的 TODO。

⚠️ **陷阱预警 `gofmt` 会改注释缩进。** 本次 `gofmt -l` 一开始报 `kv.go`，因为注释里的 tab 层级 不符合 gofmt 的格式；调整后 `gofmt -l` 才无输出。提交前跑一次。

4.4. 关键点

📌 **核心要点** 框架已经落到 `internal/db/kv.go`： `KV` 类型注释、五个方法、 `checkRI` 都有 `go doc` 可见的骨架；具体条款全部留成 `TODO(user)` 。测试和 vet 仍然通过，因为这次只改注释，没有改实现或测试。

方法	填 TODO 时必须回答的问题
`Get`	未命中时 `val` 是 nil 还是空切片？ `ok` 与 `err` 各表达什么？返回值是否别名 rep?
`Set`	`updated` 何时 true / false? 同键同值算不算更新？空键 / 空值合法吗？入参要不要拷贝？
`Del`	`deleted` 表示键存在还是删除成功？不存在时返回什么？重复删除呢？
`Open`	必须先 Open 吗？重复 Open 会清空吗？error 何时非 nil?
`Close`	Close 后还能用吗？幂等吗？error 何时非 nil?

4.5. 变式检验

变式 1： `go doc ./internal/db KV` 不显示类型注释，先查什么？ 先看注释和 `type KV` 之间有没有空行或其他注释块；再看第一行是不是以 `KV` 开头。本次就是 `// var (...)` 块夹在中间，移走后才显示。

变式 2：填 `Set` 的 `updated` 时，发现测试只覆盖新键，怎么办？ 先定契约：同键同值返回 true 还是 false。按讲义的方向是 false；定了以后， 当前实现（写后比较恒真）需要改，测试的阶段末清单也要记下「同键同值 / 不同值」。不要因为测试没测就留一个含混的 TODO。

变式 3： `TODO` 填完与测试冲突，改哪边？ 先判断哪边是合同。合同是客户依赖的行为，测试是合同的验收；冲突时先改合同， 再同步测试。改测试迁就实现，等于把验收单改成能过。

变式 4：不想保留 `// var (...)` 注释块，怎么处理？ 如果确定不用哨兵错误，就删掉整块，并在 `Get` / `Set` / `Del` 的 error 条款里写「当前实现不产生错误」。如果保留，就说明它们只在什么场合使用；不要让注释块 悬在类型注释和声明之间。

4.6. 同类归类

体系	规格写在哪	本项目对应
Go doc	紧邻声明的 `//` 注释；第一行是摘要	本次创建的框架
Javadoc	`/** ... */` + `@param` / `@return`	不要直译；Go 没有这些标签

6.031	requires / effects / returns + AF / RI	AF / RI / Safety 段落 + 方法 TODO

真实项目的注释都遵守同一形状：`sync.Map.Load` 第一行 `Load returns ...`， `bytes.Clone` 第一行 `Clone returns ...`，后面才是条款。形状一致， `go doc` 才能把它们排成可读的文档。

4.7. Go 语言机制补充

Doc comment 的关联规则。 `go/doc` 只把紧邻声明的注释当作 doc comment；中间有空行或另一个注释块，就会断开。`go doc <包> <符号>` 显示摘要与正文， `go doc -all <包>` 显示所有导出符号的完整注释。

第一行摘要。 `go doc` 把第一句（到第一个空行）作为摘要；所以第一行要以符号名开头， 并且是一句完整的话。后续段落可以写前置条件、返回值、错误、并发安全。

`gofmt` 与注释。 `gofmt` 不重写文字，但会规范化注释中的缩进层级； `gofmt -l <file>` 有输出就说明需要格式化。提交前应无输出。

`TODO(user)` 不是编译器指令。 它只是社区惯例， `go vet` 不会检查， `go test` 也不会因为它失败。它的价值是让「还没决定」在代码里显式可见。

6.031 对照。 0101 对应 R06 规格、R11 AF/RI、R08 不可变。Go 的 doc comment 是这些概念的载体：R06 的前置 / 后置条件写在方法注释里，R11 的 AF / RI 写在类型注释里，R08 的不可变与别名写在 Safety 里。

4.8. 本次操作与当前状态

已做的文件操作：

1. 修改 `internal/db/kv.go`，只加注释与移动注释块，没有改任何方法体或签名；
2. `KV` 类型注释：保留原有 AF / RI / Safety 草稿，各加一个 `TODO(user)`；
3. 把注释掉的 `// var (...)` 块从 `type KV` 之前移到 `type KV` 之后， 让类型注释紧邻声明；
4. `Open` / `Close` / `Get` / `Set` / `Del` 各加 doc comment 骨架， 第一行以方法名开头，细节留 `TODO(user)`；
5. `checkRI` 注释块里加 `TODO(user)`；
6. 按 `gofmt` 调整注释缩进。

验证结果：

1. `gofmt -l internal/db/kv.go` → 无输出；
2. `go doc ./internal/db KV` → 显示 `KV is ...` 与 AF / RI / Safety / TODO；
3. `go doc ./internal/db KV.Get` → 显示方法摘要与 TODO；
4. `go test ./internal/db/ -count=1` → `ok`；
5. `go vet ./internal/db/` → 无输出；
6. 未修改 `internal/db/kv_test.go`，未新增测试。

需要你自己动手的：

1. 逐个把 `TODO(user)` 换成具体条款；每填完一个方法，跑一次 `go doc` 看摘要是否通顺；
2. 填 `Set` 的 `updated` 时，先定同键同值的语义，再决定是否改实现；
3. 填 RI 时，先决定空键 / 空值是否合法，再决定是改实现还是删约束；
4. 填完后同步检查 `kv_test.go` 的 strategy，把「已覆盖 / 阶段末」写清楚；
5. 最后跑 `gofmt -l`、`go test`、`go vet` 各一次。

Index of Listings

Listing 1 盘点技能目录与 Delve 关键词分布 4
Listing 2 golang-troubleshooting 中的 Delve 命令清单（节选） 5
Listing 3 golang-troubleshooting 的依赖与作用路径声明 5
Listing 4 条件断点、每 100 次命中停一次、每个 goroutine 命中超过 3 次才停 11
Listing 5 命名断点后，条件和自动动作都用名字引用 11
Listing 6 断点命中时自动打印 key；再把它变成不停止的 tracepoint；每次停下都显示 len(val) 11
Listing 7 按函数名或正则批量下 tracepoint；命中只报告，不暂停 11
Listing 8 不进入交互会话，直接跑测试并报告匹配函数的调用 12
Listing 9 监视变量 count 的写入；或监视一个具体地址的读写 12
Listing 10 只看用户 goroutine、带 20 层栈、按 channel 过滤，再切换过去看栈 12
Listing 11 放大打印上限，按十六进制打印，查看类型和带类型的局部变量 13
Listing 12 先自己关优化编译测试二进制，再用 dlv exec 调试 13
Listing 13 kv.go 第 10 行的 AF 18
Listing 14 kv.go 第 13–14 行的 RI 18
Listing 15 kv.go 第 29–32 行：空的 checkRI 18
Listing 16 kv.go 第 49–51 行：先写，再拿刚写进去的值和入参比较 19
Listing 17 写入后比较恒真；updated 永远为 true 19
Listing 18 kv_test.go 第 7–20 行的 strategy 现状 20
Listing 19 kv_test.go 第 47–51 行的结构：删除 k1，却回头查 xxx 20
Listing 20 go doc -src sync.Map.Load：未命中不是错误 24
Listing 21 go doc -src bytes.Clone：未命中的 nil 语义可以直接引用 24
Listing 22 go doc ./internal/db KV 的输出：类型规格已经可见 26
Listing 23 go doc ./internal/db KV.Get 的输出：方法规格已经可见 27